

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 03 -

PADRÕES DE ESCALA HORIZONTAL EM ARQUITETURAS CLOUD

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	10
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMANDA

O QUE VEM POR AÍ?

Nas aulas anteriores, “montamos o palco” para a escalabilidade: entendemos os conceitos fundamentais e posicionamos um LoadBalancer inteligente para gerenciar nossa frota de servidores. Mas agora, precisamos olhar para dentro da nossa aplicação. De que adianta ter uma frota de carros idênticos se cada um deles só funciona com um motorista específico? E o que acontece se uma parte do nosso sistema, como o processamento de pagamentos, for muito mais lenta que o resto? Adicionar mais servidores para tudo pode ser um desperdício imenso de recursos.

Nesta aula, vamos mergulhar no coração do software e descobrir os padrões de arquitetura que permitem que uma aplicação seja verdadeiramente elástica e resiliente. Vamos começar com a "regra de ouro" da escala horizontal, uma característica sem a qual todo o resto se torna extremamente difícil. Em seguida, exploraremos os padrões que empresas como Netflix e iFood utilizam para desacoplar seus sistemas, absorver picos de demanda e garantir que uma falha em um pequeno componente não derrube todo o ecossistema. Prepare-se para pensar como um(a) arquiteto(a) de software em nuvem.

HANDS ON

Nesta seção, vamos realizar um "hands-on arquitetural", onde o foco não será em escrever longos blocos de código, mas em analisar trechos de configuração e pseudocódigo que materializam os padrões discutidos. O objetivo é que você consiga identificar esses padrões no mundo real e entenda como eles são implementados nas ferramentas que o mercado utiliza. As videoaulas correspondentes guiarão você por essa exploração visual e conceitual.

Começaremos analisando a diferença prática entre uma aplicação stateless e stateful. Mostraremos exemplos de código simples para ilustrar como o armazenamento de estado da sessão na memória do servidor impede a livre distribuição de carga por um LoadBalancer, enquanto uma abordagem stateless, que externaliza o estado, resolve esse problema. Em seguida, vamos explorar os arquivos de configuração que definem as "unidades de escala", como um ReplicaSet em um arquivo YAML do Kubernetes e as configurações de um Auto ScalingGroup no console da AWS.

Para finalizar, faremos um exercício de design. Vamos desenhar juntos, uma arquitetura simples baseada no padrão de Workers e Filas. Usando pseudocódigo, demonstraremos como uma aplicação "produtora" envia uma tarefa para uma fila, como o AWS SQS, e como uma aplicação "trabalhadora" (worker) consome, processa e remove essa tarefa da fila. Este exercício prático consolidará seu entendimento sobre como o desacoplamento funciona e como ele permite escalar diferentes partes de um sistema de forma independente.

SAIBA MAIS

Ter uma infraestrutura elástica é apenas metade da batalha. Para que a escala horizontal funcione de maneira eficaz e previsível, a própria aplicação precisa ser projetada seguindo um princípio fundamental, uma verdadeira "regra de ouro": ela deve ser stateless (sem estado). Uma aplicação stateful é aquela que armazena informações da sessão do cliente na memória do próprio servidor onde está sendo executada. Um exemplo clássico é um carrinho de compras cujos itens são guardados em uma variável de sessão no servidor. Isso cria um problema sério para a escalabilidade: o cliente fica "preso" àquele servidor específico. Se o LoadBalancer o enviar para outro servidor da frota o contexto da sua sessão, seu carrinho de compras será perdido. Para contornar isso, seria necessário usar sticky sessions, uma prática que, como vimos, limita a resiliência e a eficiência do balanceamento.

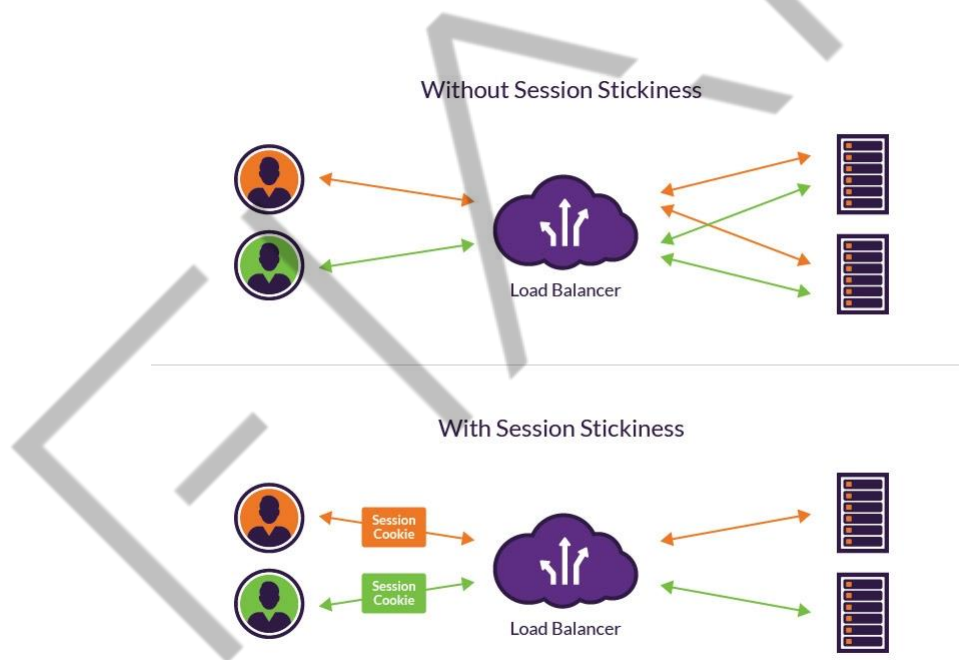


Figura 1 - Chamada com Stick session

Fonte: <https://www.imperva.com/learn/availability/sticky-session-persistence-and-cookies/>(s.d)

Por outro lado, uma aplicação stateless não guarda nenhuma informação de contexto do cliente entre uma requisição e outra. Cada requisição é tratada de forma independente e contém toda a informação necessária para ser processada. O estado da sessão, como os itens do carrinho de compras, é externalizado para um serviço de armazenamento compartilhado e de alta velocidade, como um banco de

dados ou um cache distribuído (ex: Redis). Dessa forma, qualquer servidor da frota pode atender a qualquer requisição de qualquer cliente a qualquer momento, tornando a aplicação perfeitamente compatível com a escala horizontal e o failover automático.

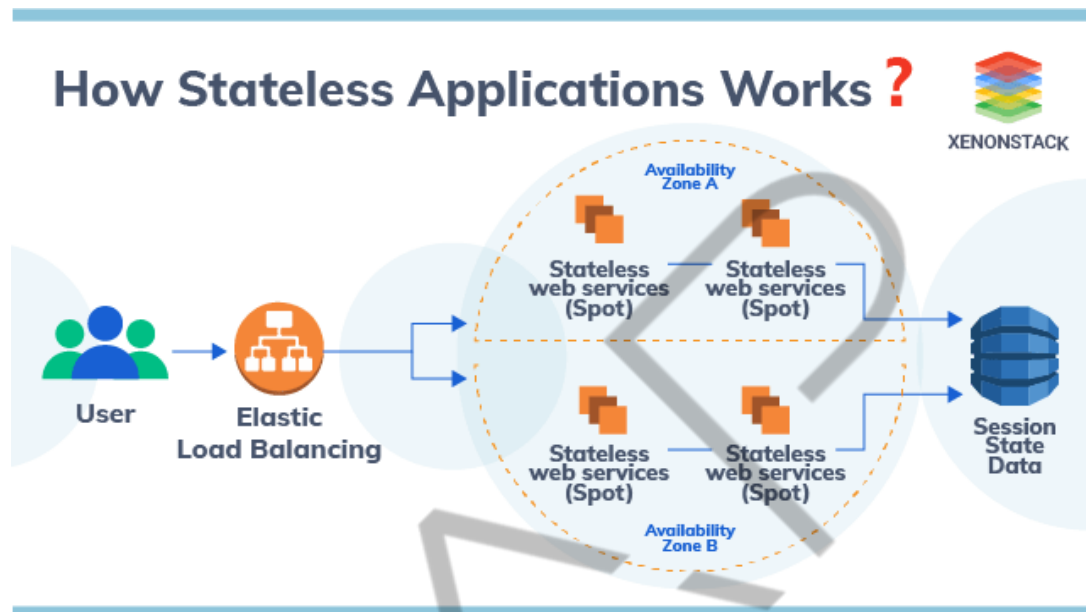


Figura 2 - Como Uma aplicação Stateless Trabalha

Fonte: <https://www.xenonstack.com/insights/stateful-and-stateless-applications>(s.d)

Uma vez que temos uma aplicação stateless, podemos gerenciá-la com as unidades de escala fornecidas pelos ambientes de nuvem e orquestradores. No mundo do Kubernetes, essa unidade é o ReplicaSet, um objeto que garante que um número especificado de cópias idênticas de um Pod (a menor unidade de execução do Kubernetes) esteja sempre em execução. No mundo de provedores de nuvem como a AWS, o conceito análogo é o Auto ScalingGroup (ASG), que gerencia o ciclo de vida de um grupo de máquinas virtuais (instâncias EC2), garantindo que a quantidade desejada de instâncias esteja sempre saudável e disponível.

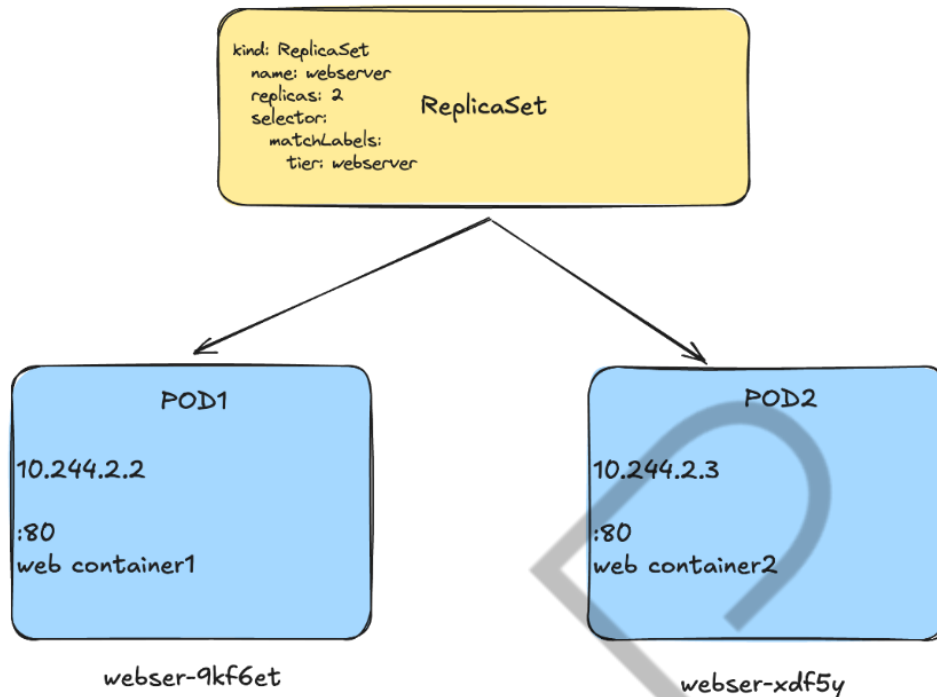


Figura 3 - ReplicaSet

Fonte: <https://unicast.com.br/posts/kubernetes-101-replicaset/> (2024)

Um dos padrões mais poderosos para criar sistemas desacoplados e escaláveis é o de Workers e Filas. Imagine um e-commerce durante a Black Friday. O processo de "confirmar o pedido" envolve várias etapas: validar o pagamento, dar baixa no estoque, emitir a nota fiscal, enviar um e-mail de confirmação. Algumas dessas etapas podem ser lentas. Se tudo for feito de forma síncrona, o usuário ficaria esperando na tela por um longo tempo.

O padrão de Workers e Filas resolve isso. A aplicação principal, ao receber o pedido, atua como um "Produtor": ela apenas cria uma "mensagem" com os detalhes do pedido e a publica em uma fila (um serviço como AWS SQS ou RabbitMQ). Em seguida, ela retorna uma resposta imediata ao usuário ("Seu pedido foi recebido e está sendo processado!"). Do outro lado, um grupo separado de aplicações, os "Workers" ou "Consumidores", fica constantemente monitorando a fila, pegando as mensagens uma a uma e realizando o trabalho pesado de forma assíncrona.

A grande vantagem é que podemos escalar o número de workers de forma independente da aplicação principal, com base no tamanho da fila, garantindo que os picos de demanda sejam absorvidos sem impactar a experiência do usuário.

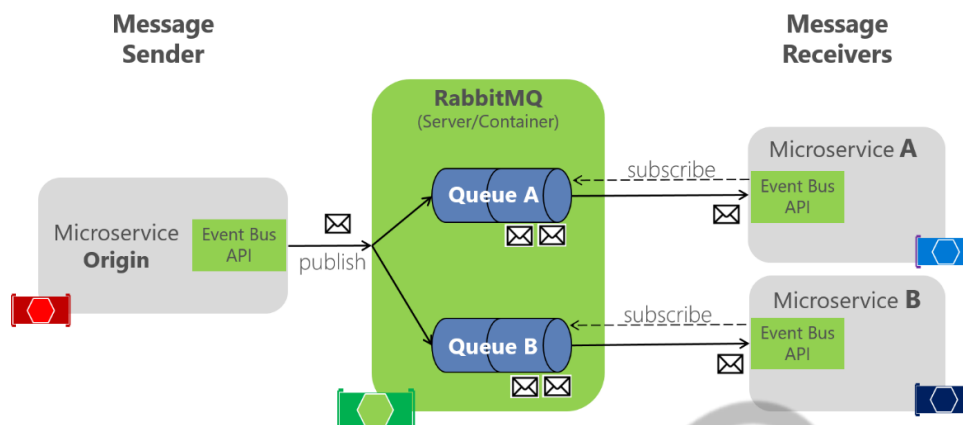


Figura 4 - Implementando Rabbit MQ
Fonte: [Microsoft](#) (2025)

À medida que as arquiteturas se tornam mais complexas, especialmente com microsserviços, outros padrões de design avançados se tornam essenciais. O padrão Sidecar, por exemplo, consiste em acoplar um contêiner auxiliar ao contêiner principal da aplicação dentro do mesmo Pod no Kubernetes. Esse sidecar pode lidar com tarefas transversais como coleta de logs, monitoramento ou atuar como um proxy de rede, sem que o código da aplicação principal precise saber disso.

O padrão Ambassador é uma variação que atua como um proxy focado em simplificar a comunicação com o mundo exterior ou com outros serviços, lidando com detalhes como tentativas (retries) e descoberta de serviços. Já o Sharding é um padrão focado em dados, representando a escala horizontal para bancos de dados. Em vez de um único banco de dados monolítico, os dados são particionados (divididos) em múltiplos bancos de dados menores e independentes (os shards), permitindo que a carga de escrita e leitura seja distribuída.

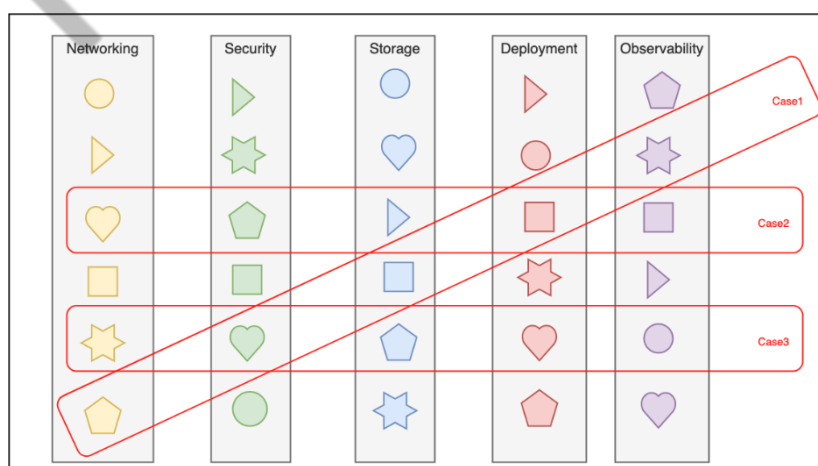


Figura 5 - Complexidade do Microserviço

Fonte: <https://erkanerol.github.io/post/complexity-of-kubernetes/> (2022)

Naturalmente, a implementação desses padrões traz consigo importantes trade-offs. A discussão sobre Custo vs. Performance no scaling automático é central. Uma política de autoscaling agressiva, que adiciona servidores ao primeiro sinal de aumento de carga, garante a melhor performance para o usuário, mas pode resultar em custos de nuvem elevados. Uma política conservadora economiza dinheiro, mas pode levar à degradação do serviço durante picos repentinos.

A disciplina de FinOps surge para buscar o equilíbrio nessa equação. Além disso, em sistemas distribuídos, a falha é uma certeza. A Resiliência é projetada por meio de estratégias como Retry (tentar novamente uma operação que falhou, idealmente com um tempo de espera crescente, conhecido como exponentialbackoff) e Fallback (ter um plano B quando as tentativas falham, como servir dados de um cache ou retornar uma resposta padrão em vez de um erro).

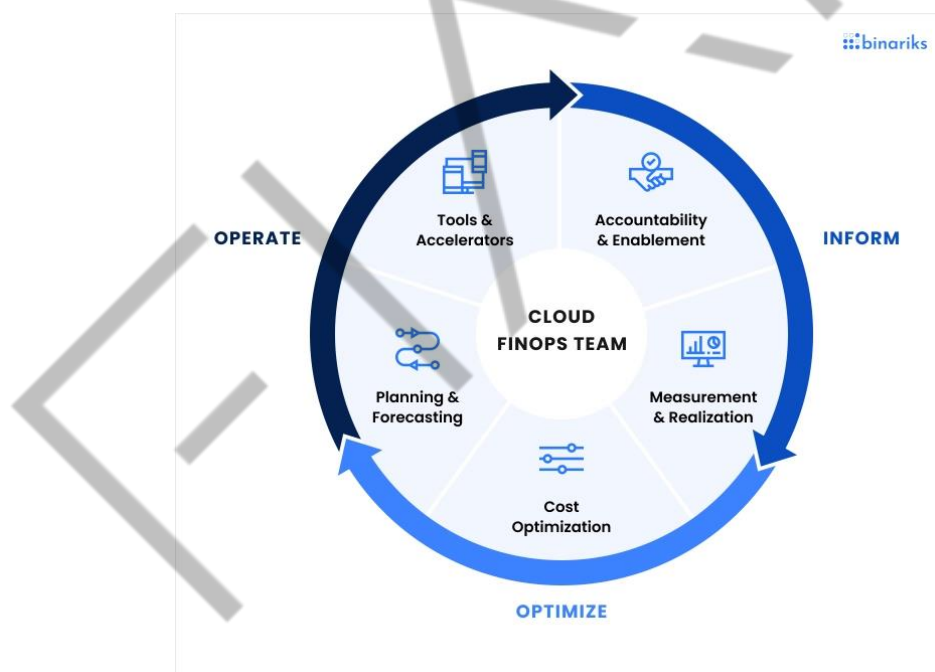


Figura 6 - Custo vs Performance

Fonte: <https://binariks.com/blog/finops-cloud-cost-optimization/> (2024)

O mundo real está repleto de aplicações desses padrões. Empresas como Netflix, Instagram e iFood são mestres na arte da escalabilidade horizontal. A Netflix utiliza uma arquitetura massiva de microsserviços com padrões de resiliência sofisticados para garantir que a falha de um serviço de recomendação não impeça você de assistir a um filme. O Instagram e o iFood usam intensivamente o padrão de

workers e filas para processar bilhões de uploads de fotos e milhões de pedidos de forma assíncrona, mantendo a interface do aplicativo sempre ágil.

Essa conexão com o mundo real se reflete nas ferramentas modernas que automatizam esses conceitos. Ferramentas como o KEDA (Kubernetes Event-driven Autoscaling) são a personificação do escalonamento do padrão de Workers e Filas. O KEDA pode monitorar diretamente o tamanho de uma fila no AWS SQS ou o lag de um tópico no Kafka e escalar o número de workers de zero a N, de acordo com a demanda real, otimizando drasticamente os custos. Já o Karpenter, um provisionador de nós para Kubernetes, ataca diretamente o problema do Custo vs. Performance. Em vez de gerenciar grupos fixos de servidores, ele observa as necessidades das aplicações e provisiona a infraestrutura computacional exata, mais barata e mais adequada para aquela carga de trabalho, naquele exato momento, elevando a eficiência da nuvem a um novo patamar.

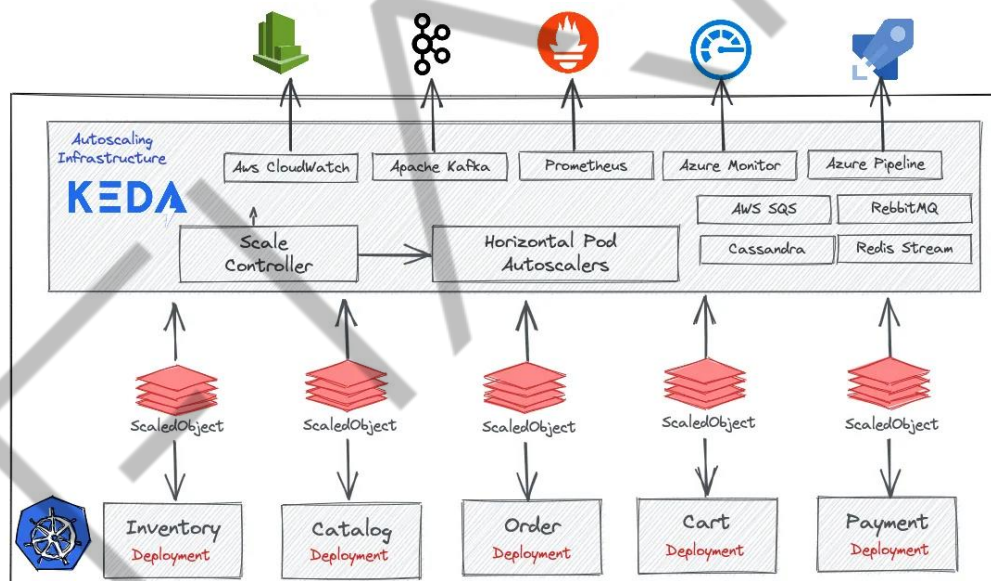


Figura 7 - Keda (Kubernetes Event-driven Autoscaling)

Fonte: <https://www.redhat.com/en/blog/event-driven-autoscaling-kubernetes> (2023)

MERCADO, CASES E TENDÊNCIAS

Os padrões de arquitetura em nuvem não são apenas teorias; eles são a base que sustenta as operações das maiores empresas de tecnologia do mundo e ditam as tendências do mercado.

Case: A Arquitetura de Resiliência da Netflix

A Netflix é um dos maiores estudos de caso em arquitetura de microsserviços e resiliência. Para garantir que seu serviço de streaming nunca pare, eles popularizaram o padrão "CircuitBreaker" por meio de sua biblioteca Hystrix. Quando um serviço começa a falhar repetidamente, o circuitbreaker "abre", impedindo que novas requisições sejam enviadas para o serviço falho e redirecionando para um fallback. Isso evita as temidas falhas em cascata e é um exemplo prático de implementação dos padrões de resiliência que discutimos.

Tendência: A Ascensão da Arquitetura Serverless

A arquitetura Serverless, com serviços como o AWS Lambda, pode ser vista como a evolução máxima do padrão de Workers e Filas. Em vez de gerenciar uma frota de workers, pessoas desenvolvedoras simplesmente escrevem o código (a função) que processa um evento (a mensagem). A plataforma de nuvem se encarrega de escalar de zero a milhares de execuções concorrentes de forma automática e transparente, baseada na chegada de eventos. Essa tendência simplifica radicalmente a operação e otimiza os custos, pois o pagamento é feito por execução, e não por tempo de servidor ocioso.

Notícia: O Papel do Service Mesh na Gestão de Padrões

Com a proliferação de microsserviços, a complexidade da comunicação entre eles cresceu. Notícias e artigos sobre a adoção de "Service Meshes" como Istio e Linkerd estão cada vez mais comuns. Essas ferramentas implementam os padrões Sidecar e Ambassador de forma padronizada em toda a infraestrutura. Elas fornecem, de forma transparente para a aplicação, funcionalidades de resiliência (retries, timeouts), segurança e observabilidade, tornando a gestão de arquiteturas distribuídas complexas uma tarefa mais gerenciável e padronizada.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, nos aprofundamos na camada de software, descobrindo os padrões de arquitetura essenciais para a escala horizontal. Você aprendeu a "regra de ouro" da escalabilidade, a importância de aplicações stateless, e entendeu como as unidades de escala, como ReplicaSets e Auto ScalingGroups, gerenciam nossa frota de aplicações. Exploramos o poderoso padrão de Workers e Filas para desacoplamento e absorção de picos, e fomos apresentados a padrões avançados como Sidecar, Ambassador e Sharding.

Discutimos os trade-offs críticos entre custo e performance e a necessidade de projetar para a falha com estratégias de resiliência. Por fim, conectamos toda essa teoria ao mundo real, vendo como grandes empresas aplicam esses conceitos e como ferramentas modernas como KEDA e Karpenter os automatizam.

REFERÊNCIAS

KARPENTER. **KarpenterDocumentation**. Disponível em: <https://karpenter.sh/docs/>. Acesso em: 28 ago. 2025.

KEDA. **KEDA Documentation**. Disponível em: [link suspeito removido]. Acesso em: 28 ago. 2025.

NETFLIX TECHNOLOGY BLOG. **Netflix Technology Blog**. Disponível em: <https://netflixtechblog.com/>. Acesso em: 28 ago. 2025.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2015.

PALAVRAS-CHAVE

Keda. Kubernetes. Microserviço. Stick Session. Finops.

FINOPS



POS TECH